REVERSTEEM

White Paper

Reversi on Steem Blockchain

Version 0.1 • App Identifier: `reversteem/0.1`

Author
`@puncakbukit`

Published
March 2, 2026

Live Demo: https://puncakbukit.github.io/reversteem/

Table of Contents

Table of Contents 1
Abstract 4
Keywords 4
1. Overview 4
2. App Identifier 5
3. On-Chain Game Model 5
3.1 Blockchain Objects 5
3.2 Replay Order 5

4. JSON Metadata Schemas 7
4.1 Game Start (Root Post) 7
Field Reference 7
4.2 Join Comment 7
4.3 Move Comment 8
Field Reference 8
Board Index Layout 8
4.4 Timeout Claim Comment 9
5. Game Replay Algorithm 10
5.1 Initialisation 10
5.2 Processing Events (in chronological order) 10
Join Comment 10
Move Comment 10
Timeout Claim Comment 10
5.3 End-of-Game Detection 11
5.4 Automatic Pass Rule 11
6. Move Validation 11
6.1 Flip Calculation 11
6.2 Validation Checklist 11
7. Timeout System 13
7.1 Time Limit Parameters 13
7.2 Clock Mechanics 13
7.3 Claiming Timeout 13
7.4 Time Control Presets 13
8. ELO Rating System 14
8.1 Parameters 14
8.2 Formula 14
8.3 Deterministic Reconstruction 14
8.4 Automatic Game Tagging 15
9. Invite System 16
10. Replay Caching 16
11. Security Model 17
12. Multi-RPC Fallback 17
13. Steem Keychain Integration 18
13.1 requestPost call signature 18
13.2 Permlink generation 18
13.3 Login flow 18
14. Post Body Format 19

14.1 Game Start body 19
14.2 Move body 19
14.3 Timeout Claim body 19
15. Spectator Console 20
16. Implementation Notes 20
16.1 Third-Party Libraries 20
16.2 Hash-Based Routing 20
16.3 Polling 21
16.4 Reply Classification 21
17. Glossary 22

Abstract

Abstract

Reversteem is a fully decentralized, serverless implementation of Reversi (Othello) built on the Steem blockchain. This white paper describes the Reversteem protocol: a lightweight, open standard for encoding turn-based board games as immutable event logs in a blockchain's native post-and-comment system — without smart contracts, without a backend server, and without any centralized authority.

Game state is derived entirely in the browser by replaying on-chain history in chronological order. A strict JSON metadata schema, sequential move numbering, and deterministic replay rules ensure tamper-resistance: no client can forge a game outcome, and every spectator independently verifies the result.

The protocol additionally specifies a claimable per-move timeout system, an invite-based access control mechanism, and a fully client-derived ELO rating system — all computed locally from on-chain data with no external dependencies.

Reversteem demonstrates that complete, competitive multiplayer games are viable on a social blockchain using only posts and comments as a data layer.

Keywords

Steem blockchain, Reversi, Othello, decentralized game, serverless dApp, *deterministic replay*, *blockchain event log*, ELO rating, Steem Keychain, Vue 3, static frontend, turn-based game protocol.

1. Overview

Reversteem is a fully decentralized, serverless Reversi (Othello) game running on the Steem blockchain. There is no backend server, no database, and no smart contract. The blockchain's native post-and-comment system acts as an immutable event log. All game state — boards, scores, ratings, timeouts — is computed entirely in the browser by replaying on-chain history.

Core Properties

- ✓ 100% static frontend (GitHub Pages compatible)
- ✓ No backend server, no database, no smart contracts
- ✓ Deterministic board reconstruction from blockchain history
- ✓ Strict JSON metadata filtering by app identifier
- ✓ Per-move time limits claimable via on-chain timeout claims
- ✓ Client-derived ELO ratings cached in localStorage
- ✓ Multi-RPC automatic fallback across 4 public nodes

2. App Identifier

Every on-chain interaction produced by Reversteem must include an app identifier in the json_metadata field. This is the primary filter used during replay to distinguish Reversteem protocol messages from all other Steem posts and comments.

Field	Value
app identifier	`reversteem/0.1`
Format	`"app": "reversteem/{version}"`
Filter rule	`app.startsWith("reversteem/")`

Any comment whose json_metadata does not satisfy the filter is treated as a spectator comment and is displayed in the chat console. It has no effect on game state.

3. On-Chain Game Model

A Reversteem game maps to a single Steem post thread. The post itself represents the game, and all game actions are encoded as replies to that post (or replies to replies).

3.1 Blockchain Objects

Object	Meaning
`Root Post`	Represents the game. Author = Black player. Created by Black at game start.
`Join Comment`	Registers the White player. Posted as a reply to the root post.
`Move Comment`	Represents one move on the board. Posted as a reply to the root post.
`Timeout Claim`	Claims victory after the opponent exceeds the per-move time limit.
`Spectator Comment`	Any other reply. Displayed in chat console, ignored for game state.

3.2 Replay Order

All replies to a game's root post are sorted chronologically by their created timestamp before replay. Steem timestamps omit the UTC suffix (e.g., "2024-01-01T12:00:00"); the client appends 'Z' to force UTC interpretation and avoid timezone-offset errors.

```
// Correct UTC parsing of Steem timestamps
```

```
function steemDate(ts) {
  if (typeof ts === 'string' && !ts.endsWith('Z')) ts += 'Z';
  return new Date(ts);
}

replies.sort((a, b) => steemDate(a.created) - steemDate(b.created));
```

4. JSON Metadata Schemas

All game events are encoded in the json_metadata field of Steem posts and comments. The following schemas define the protocol wire format.

4.1 Game Start (Root Post)

Posted by the Black player to create a new game. This is the game's root post.

```
{
  "app":             "reversteem/0.1",
  "type":            "game_start",
  "black":           "<steem-username>",
  "white":           null,
  "timeoutMinutes": 60,
  "status":          "open",
  "invites":         ["alice", "bob"],   // optional — omit for open game
  "tags":            ["standard", "elo-1200", "reversi", "othello", "board", "game",
"steem"]
}
```

Field Reference

Field	Type	Description
app	string	Always "reversteem/{version}". Required for protocol filtering.
type	string	"game_start" — identifies this as a game creation event.
black	string	Steem username of the Black player (also the post author).
white	null	Always null at creation. White player is determined by Join Comment.
timeoutMinutes	integer	Per-move time limit in minutes. Min: 1, Max: 10080 (7 days), Default: 60.
status	string	"open" at creation. Informational only — not used in replay.
invites	string[]	Optional. If present, only listed usernames may join. Empty/absent = open game.
tags	string[]	Steem post tags. Auto-generated. First tag = time preset or mins-N. Second = elo-N.

4.2 Join Comment

Posted by a non-Black player to register as the White player. Must be a direct reply to the root post. First valid join wins; subsequent join comments are ignored.

```
{
  "app":    "reversteem/0.1",
  "action": "join"
}
```

Validation rules applied during replay:

- Author must not be the Black player.
- No White player must already be registered.
- If invites is set on the root post, author.toLowerCase() must appear in the invites array.
- Clock (lastMoveTime) is set to this comment's created timestamp when accepted.

4.3 Move Comment

Posted by the active player to place a disc on the board.

```
{
  "app":        "reversteem/0.1",
  "action":     "move",
  "index":      28,
  "moveNumber": 4
}
```

Field Reference

app	`"reversteem/0.1"`
action	`"move"`
index	`Integer 0–63. Board cell index in row-major order (A1=0, H8=63).`
moveNumber	`Zero-based sequential counter. Must equal appliedMoves at replay time.`

Board Index Layout

The board is a flat array of 64 cells, indexed 0 (top-left, A1) to 63 (bottom-right, H8) in row-major order.

```
   A    B    C    D    E    F    G    H
1 |  0 |  1 |  2 |  3 |  4 |  5 |  6 |  7 |
2 |  8 |  9 | 10 | 11 | 12 | 13 | 14 | 15 |
3 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 |
4 | 24 | 25 | 26 |[27]|[28]| 29 | 30 | 31 |   ← initial white/black
5 | 32 | 33 | 34 |[35]|[36]| 37 | 38 | 39 |   ← initial black/white
```

```
6| 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 |
 |----|----|----|----|----|----|----|----|
7| 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 |
 |----|----|----|----|----|----|----|----|
8| 56 | 57 | 58 | 59 | 60 | 61 | 62 | 63 |
 |____|____|____|____|____|____|____|____|
```

4.4 Timeout Claim Comment

Posted by the non-active player to claim victory when the opponent has exceeded the per-move time limit.

```
{
  "app":           "reversteem/0.1",
  "action":        "timeout_claim",
  "claimAgainst": "black",
  "moveNumber":   12
}
```

app	`"reversteem/0.1"`
action	`"timeout_claim"`
claimAgainst	`"black" or "white". The color of the player who timed out.`
moveNumber	`Must equal appliedMoves at the time of the claim.`

5. Game Replay Algorithm

Game state is never stored — it is always derived by replaying the blockchain history from scratch. This guarantees determinism: any client replaying the same event log will compute identical state.

5.1 Initialisation

- Start with the standard Reversi opening position:
  - board[27] = white (D4)
  - board[28] = black (E4)
  - board[35] = black (D5)
  - board[36] = white (E5)
- appliedMoves = 0
- turn = "black" (Black always moves first)
- lastMoveTime = rootPost.created (updated to join time when White joins)
- finished = false, winner = null

5.2 Processing Events (in chronological order)

For each reply, sorted by created timestamp:

Join Comment

- Accepted only if: no whitePlayer yet, author ≠ blackPlayer, and author is in invites list (if set).
- Sets whitePlayer and gameStartTime = reply.created.
- lastMoveTime is updated to gameStartTime — the clock starts when both players are present.

Move Comment

- move.moveNumber must equal appliedMoves (sequential, no gaps, no duplicates).
- If current turn player has no valid moves, pass to opponent; if neither has moves, game ends.
- move.author must equal the expected player for the current turn color.
- move.index must be in range 0–63.
- getFlipsForBoard(board, index, turn) must return at least one flip.
- On success: apply flips, increment appliedMoves, update lastMoveTime, swap turn.

Timeout Claim Comment

- Accepted only if the game is not already finished.
- claim.moveNumber must equal appliedMoves.
- appliedMoves must be > 0 (first move is exempt from timeout).
- (steemDate(claim.created) − steemDate(lastMoveTime)) / 60000 ≥ timeoutMinutes.

- claim.claimAgainst must equal the current turn color.
- claim.author must equal the opponent of the timed-out player.
- On success: finished = true, winner = opponent color.

5.3 End-of-Game Detection

After all events are processed:

- If neither player has any valid move → finished = true.
- winner = "black" if score.black > score.white, "white" if score.white > score.black, "draw" otherwise.
- If finished = true, currentPlayer is set to null; the board is frozen.

5.4 Automatic Pass Rule

If the player whose turn it is has no valid moves, the turn is automatically passed to the opponent — no on-chain pass transaction is required or supported. This is enforced silently during replay.

6. Move Validation

6.1 Flip Calculation

The eight search directions from a given cell index are:

```
const DIRECTIONS = [-8, 8, -1, 1, -9, -7, 7, 9];
//                   N   S   W   E  NW  NE  SW  SE
```

For each direction, the flip algorithm walks outward collecting opponent cells. If it reaches a cell belonging to the active player (not the opponent), all collected cells are valid flips. An edge-wrap guard (isOnBoardGeneric) prevents crossing the left or right board boundary.

```
function getFlipsForBoard(board, index, player) {
  if (board[index]) return [];      // cell must be empty
  let all = [];
  for (const dir of DIRECTIONS) {
    all = all.concat(collectFlipsForBoard(board, index, dir, player));
  }
  return all;  // empty → invalid move
}
```

6.2 Validation Checklist

A move is accepted during replay only when ALL of the following conditions are true:

#	Condition	Purpose
1	`move.moveNumber === appliedMoves`	Sequential ordering; prevents gaps and duplicates
2	`Both players are present (whitePlayer set)`	Game has not started yet
3	`move.author === expected player for turn color`	Enforces turn order
4	`move.index in range 0–63`	Valid board position
5	`getFlipsForBoard(...).length > 0`	Move must flip at least one disc
6	`Game is not finished`	No moves after game end
7	`No valid timeout claim is pending`	Once timeout threshold passed, only claims accepted

Invalid moves are silently skipped. They leave no trace in the derived game state.

7. Timeout System

7.1 Time Limit Parameters

Default	`60 minutes`
Minimum	`1 minute`
Maximum	`10,080 minutes (7 days)`
Stored in	`timeoutMinutes field of root post json_metadata`

7.2 Clock Mechanics

The clock measures elapsed time between the last accepted event and the current moment. Clock semantics:

- Clock starts when White joins (lastMoveTime = join comment's created timestamp).
- Clock resets on each accepted move (lastMoveTime = move comment's created timestamp).
- The first move (appliedMoves === 0) is exempt from timeout. Black cannot be timed out before they have had a chance to notice White has joined.

```
const minutesPassed =
  (steemDate(claim.created) - steemDate(lastMoveTime)) / (1000 * 60);

const isTimeout = minutesPassed >= timeoutMinutes;
```

7.3 Claiming Timeout

Timeout is claimable — it does not trigger automatically. The opponent must post a timeout_claim comment. This comment is validated during replay. If valid, the game ends immediately and no further moves can be applied.

Race Condition Safety

Once the timeout threshold is exceeded, all subsequent move comments by the timed-out player are rejected during replay. Only a valid timeout_claim can close the game. This prevents ambiguity between a late move and a racing claim.

7.4 Time Control Presets

Preset Name	timeoutMinutes	Auto-generated Tag
`blitz`	1	blitz
`rapid`	5	rapid

standard	60	standard (default)
daily	1440	daily
custom	N	mins-N

8. ELO Rating System

Reversteem includes a fully deterministic, client-derived ELO rating system. Ratings are computed locally from finished games and cached in localStorage. There is no on-chain rating storage and no central authority.

8.1 Parameters

Base rating	1200
K-factor	32
Cache key	reversteem_elo_cache (localStorage)
Scope	All finished games ordered chronologically by created

8.2 Formula

```
// Expected score for player A vs player B
E_A = 1 / (1 + 10 ^ ((R_B - R_A) / 400))

// New rating for player A
R_A' = R_A + K * (S_A - E_A)

// S_A (actual score)
S_A = 1.0   if A wins   (or wins by timeout)
S_A = 0.0   if A loses
S_A = 0.5   if draw
```

8.3 Deterministic Reconstruction

- Collect all finished games from the blockchain.
- Sort chronologically by created timestamp.
- Replay outcomes in order, applying ELO updates incrementally.
- Cache is incremental: only games newer than lastProcessed are computed on reload.
- Any client replaying the same history computes identical ratings.

8.4 Automatic Game Tagging

When a new game is created, tags are auto-generated from game parameters and embedded in the root post json_metadata:

Position 1	`Time preset name (blitz/rapid/standard/daily) or mins-N for custom`
Position 2	`elo-{rating} — Black player's current raw ELO (integer-rounded)`
Positions 3–7	`reversi, othello, board, game, steem (genre/discovery tags)`

9. Invite System

The Black player may optionally restrict who can join as White by specifying an invites array in the root post metadata.

- If invites is absent or empty → the game is open to any Steem user.
- If invites is non-empty → only users whose username.toLowerCase() appears in the array may post a valid Join Comment.
- Maximum 3 invited users.
- Invite enforcement occurs at two layers: the UI (guard before Keychain call) and deterministic replay (Join Comments from non-invited users are silently skipped).

```
// Root post metadata with invite restriction
{
  "app":     "reversteem/0.1",
  "type":    "game_start",
  "black":   "alice",
  "invites": ["bob", "carol"]
}
```

Security Note

Invite enforcement is always re-read from the root post at replay time — never from the replay cache. This ensures the invite list cannot be bypassed by cache poisoning.

10. Replay Caching

To avoid redundant blockchain fetches, derived game state is cached in localStorage on a per-game basis.

Cache key format	`reversteem_cache_{author}_{permlink}`
Invalidated when	`Reply count changes OR latest reply created timestamp changes`
Always re-read from root	`invites  (never trusted from cache)`
Authoritative source	`Full replay — cache is an optimisation only`

```
// Cache is valid only if both conditions match:
cache.lastCreated === latestReplyCreated
  && cache.replyCount === replies.length
```

11. Security Model

Reversteem's security properties are derived entirely from the deterministic replay model. No client authority is required.

- Strict metadata filtering — only comments with app starting with "reversteem/" are processed.
- Sequential moveNumber — prevents gaps, replay attacks, and duplicate moves.
- Author/turn enforcement — only the expected player's move is accepted.
- Legal flip validation — at least one disc must be flipped.
- Automatic pass logic — no on-chain pass transaction required or accepted.
- Deterministic timeout enforcement — once exceeded, only timeout_claim closes the game.
- End-state freeze — no moves accepted after game is finished.
- Invite enforcement — re-read from root post at every replay, never from cache.
- ELO determinism — identical history → identical ratings on any client.
- Double-move prevention — UI lock (isSubmitting) plus replay-level moveNumber guard.

Tamper Resistance

Because game state is derived from immutable Steem blockchain history:
- Malicious clients cannot forge outcomes.
- Malicious users cannot manipulate ratings.
- Every spectator independently verifies the game.

12. Multi-RPC Fallback

All blockchain API calls are wrapped in a fallback mechanism that rotates through a list of public RPC nodes on failure.

Node 1	`https://api.steemit.com`
Node 2	`https://api.justyy.com`
Node 3	`https://steemd.steemworld.org`
Node 4	`https://api.steem.fans`

On any API error, the client increments currentRPCIndex and retries transparently. No backend proxy is required. If all nodes fail, the error is surfaced to the user.

13. Steem Keychain Integration

All write operations (game creation, join, move, timeout claim, spectator comment) are signed and broadcast via the Steem Keychain browser extension. The dApp never handles or stores private keys.

13.1 requestPost call signature

```
steem_keychain.requestPost(
  username,          // Steem account name
  title,             // Post title (empty string for comments)
  body,              // Post body (Markdown)
  parentPermlink,    // Parent post's permlink (APP_NAME for root post)
  parentAuthor,      // Parent post's author (empty string for root post)
  JSON.stringify(meta),  // json_metadata with tags merged in
  permlink,          // Unique permlink for this post/comment
  "",               // Beneficiaries (unused)
  callback           // (response) => { ... }
);
```

13.2 Permlink generation

Permlinks are generated deterministically using a sanitised slug of the game title plus a millisecond timestamp suffix, ensuring global uniqueness:

```
function convertToPermlink(title) {
  const slug = title.toLowerCase().trim()
    .replace(/\s+/g, '-')        // spaces → hyphens
    .replace(/[^a-z0-9-]/g, '')  // strip invalid chars
    .replace(/-+/g, '-')          // collapse hyphens
    .replace(/^-|-$/g, '');       // trim edges
  return slug.slice(0, 241);      // max 255 chars - 14 for '-' + timestamp
}

const permlink = `${convertToPermlink(gameTitle)}-${Date.now()}`;
```

13.3 Login flow

Authentication uses Keychain's requestSignBuffer to verify account ownership without exposing private keys:

- User enters their Steem username.
- steem_keychain.requestSignBuffer is called with a challenge string signed by the Posting key.
- The verified username from the response is checked against the entered username.
- On success, the username is persisted in localStorage.

14. Post Body Format

Every game action includes a Markdown body for native Steemit compatibility. Spectators reading the Steem blockchain directly (without the dApp) can follow the game through these bodies.

14.1 Game Start body

```
## New Reversteem Game

Black: @alice
Timeout per move: 60 minutes
Invited: @bob, @carol  ← only present if invites set

### Current Board

|   | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| **4** | · | · | · | ○ | ● | · | · | · |
| **5** | · | · | · | ● | ○ | · | · | · |

---
Move by commenting via [Reversteem](https://puncakbukit.github.io/reversteem/).
```

14.2 Move body

Each move comment includes the move coordinate, a full board snapshot in Markdown, and the move transcript up to that point. If the move ends the game, a result line is appended.

```
## Move by @alice

Played at D3

### Current Board
[Markdown table with ●/○/· symbols]

### Move Transcript
| # | Color  | Square | Time |
|---|--------|--------|------|
| 1 | ● Black | D3     | ...  |

---
□ **Game Over** — ● alice wins! (34-30)   ← only on final move
```

14.3 Timeout Claim body

```
## Timeout Claim by @bob

@alice exceeded the 60 min move time limit.
```

```
---
□ **Game Over** — □ @bob wins by timeout!
```

15. Spectator Console

Any Steem comment in the game's reply tree that is not classified as a protocol event (game_start, join, move, timeout_claim) is treated as a spectator comment and displayed in the chat console.

- Spectator comments may be nested replies to move comments (anchored to the move's square coordinate) or direct replies to the root post.
- The dApp displays spectator comments in a terminal-style panel below the board, sorted chronologically.
- Authenticated users may post spectator comments via Keychain. The comment is posted as a reply to the most recent move comment, or to the root post if the game has not started.
- Spectator comments have zero effect on game state — they are purely cosmetic.

16. Implementation Notes

16.1 Third-Party Libraries

steem-js 0.8.0	`Steem blockchain API client (CDN: @steemit/steem-js)`
Vue 3	`Frontend framework (CDN: unpkg.com/vue@3)`
Vue Router 4	`SPA routing with hash history (CDN: unpkg.com/vue-router@4)`
Steem Keychain	`Browser extension for transaction signing`

16.2 Hash-Based Routing

Vue Router uses createWebHashHistory, making all routes hash-prefixed (#/). This enables hosting on GitHub Pages without server-side redirect configuration. Routes:

#/	`Dashboard — open games list`
#/game/:author/:permlink	`Live game board with polling`
#/@:user	`Profile — all games for a user`
#/leaderboard	`ELO leaderboard`
#/about	`Protocol documentation`

16.3 Polling

GameView polls the blockchain every 15 seconds using setTimeout (not setInterval). The poll is stopped when the game finishes. To prevent board flicker on each poll, game state fields are merged individually (not replaced), and account data is only re-fetched when a player's username changes.

16.4 Reply Classification

All replies to a game post are split into two buckets by classifyReplies():

- gameReplies — comments with valid reversteem/ app metadata and a recognised action (join, move, timeout_claim). These are used exclusively for game state replay.
- spectatorReplies — everything else. Displayed in the spectator console. Never used for replay.

17. Glossary

appliedMoves	Zero-based count of successfully replayed moves. Drives moveNumber validation.
Black player	The author of the root post. Always moves first.
claimAgainst	The color ("black" or "white") of the player alleged to have timed out.
currentPlayer	The color whose turn it currently is (null when game is finished).
DXA	DocumeNt eXtended Attribute. Unit of measurement in OOXML: 1440 DXA = 1 inch.
ELO	Chess rating system adapted here for Reversi. Base 1200, K=32.
finished	Boolean flag set to true when no further moves are possible or a valid timeout claim is applied.
gameStartTime	The created timestamp of the accepted Join Comment. Marks when the clock begins.
invites	Optional array of Steem usernames allowed to join as White. Empty = open game.
json_metadata	Steem post/comment field containing a JSON string with protocol data.
lastMoveTime	Timestamp of the last accepted event (join or move). Used for timeout calculation.
moveNumber	Zero-based integer in move metadata. Must equal appliedMoves to be accepted.
permlink	Unique Steem post identifier. Combined with author to address a post.
Root Post	The initial Steem post that creates the game. Author = Black player.
timeout_claim	Protocol action posted by the non-active player to claim victory after timeout.
timeoutMinutes	Per-move time limit stored in root post metadata. Min 1, max 10080.
White player	The player who joins by posting a valid Join Comment.

End of Reversteem White Paper v0.1

Source: https://github.com/puncakbukit/reversteem